

DAY 2. RAG 챗봇의 지식 베이스 만들기

Document Loader · Text Splitter · Embedding

Raw Python

직접 읽으면 왜 관리가 어려운가

Document

page_content와 metadata로 표
준화

Chunk

챗봇이 검색할 최소 단위 설계

Embedding

문장을 의미 검색 가능한 벡터로
변환

오늘 목표: 내일 만들 RAG 챗봇이 참고할 PDF 지식 베이스를 직접 준비합니다.

Day 1~5 전체 연결

Day 2는 별도 이론이 아니라 RAG 챗봇 프로젝트의 기반입니다.

Day 1

prompt | model | parser
LLM이 답변을 생성하는 최소 구조

Day 2

loader → splitter → embedding
챗봇이 참고할 지식 베이스 준비

Day 3

vectorstore → retriever → rag
CLI 기반 RAG 챗봇 v1

Day 4

retriever 비교와 개선
검색 품질을 높인 챗봇 v2

Day 5

FastAPI + HTML/CSS/JS
웹 화면이 붙은 챗봇 v3

오늘 사용할 실습 파일

실습은 Day2 실습 PPT의 파일명을 그대로 기준으로 진행합니다.

DocumentLoader

- 01_raw_documentLoader.py
- 02_DocumentLoader_TextLoader.py
- 03_DocumentLoader_CSVLoader.py
- 04_DocumentLoader_PyPDFLoader.py
- 05_DocumentLoader_WebBaseLoader.py

TextSplitter

- 06_raw_textSplitter.py
- 07_CharacterTextSplitter.py
- 08_RecursiveCharacterTextSplitter.py
- 09_RecursiveCharacterTextSplitter_pdf.py

Token / 구조 보존

- 10_token_example.py
- 11_TokenTextSplitter.py
- 12_TokenTextSplitter_korean_english.py
- 13_MarkdownHeaderTextSplitter.py
- 14_CodeSplitter.py
- 15_SemanticChunker.py

Embedding / 과제

- 16_Embedding_OpenAIEmbeddings.py
- 17_Embedding_PDF_Pages.py
- 18_Embedding_similarity_demo.py
- assignments/day2_build_knowledge_base_starter.py

시작 전 환경 확인

문서 로더, splitter, embedding은 패키지와 파일 경로가 중요합니다.

```
pip install langchain langchain-community langchain-openai \
  langchain-text-splitters python-dotenv pypdf \
  tiktoken beautifulsoup4
```

```
# SemanticChunker 선택 실습 시
pip install langchain-experimental
```

```
# .env 예시
OPENAI_API_KEY=sk-...
OPENAI_EMBEDDING_MODEL=text-embedding-3-small
```

확인할 것

가상환경이 켜져 있는가?
data 폴더에 샘플 파일이 있는가?
.env에 OPENAI_API_KEY가 있는가?
PDF 실습에는 pypdf가 필요합니다.
WebBaseLoader는 네트워크 환경에 따라 실패할 수 있습니다.

LangChain 없이 문서 읽기

- 먼저 Raw Python으로 직접 읽습니다.
- 텍스트뿐 아니라 metadata를 직접 붙여봅니다.
- 파일 형식이 늘어날 때 왜 불편한지 확인합니다.

Raw Python으로 TXT 읽기

TXT 하나만 읽을 때는 open()과 read()만으로 충분합니다.

실습 파일: 01_raw_documentLoader.py

```
from pathlib import Path

file_path = Path("./data/Sample_Text.txt")

with file_path.open("r", encoding="utf-8") as f:
    text = f.read()

print("--- Raw Python 결과 ---")
print(text)
print(type(text))
```

확인 포인트

결과 타입은 str입니다.

본문은 읽었지만 출처 정보는 없습니다.

RAG 챗봇에서 source/page를 보여주려면 별도 관리가 필요합니다.

Raw Python으로 Document 비슷한 구조 만들기

LangChain이 해주는 일을 직접 흉내 내보면 관리 부담이 보입니다.

실습 파일: 01_raw_documentLoader.py

```
raw_doc = {
    "page_content": text,
    "metadata": {
        "source": str(file_path),
        "file_name": file_path.name,
        "file_type": file_path.suffix,
    },
}

print(raw_doc["page_content"][:300])
print(raw_doc["metadata"])
```

핵심 질문

직접 만들 수는 있습니다.

하지만 이 구조는 우리가 정한 약속입니다.

- PDF는 page가 필요합니다.
- CSV는 row가 필요합니다.
- 웹은 url/title이 필요합니다.

파일 형식이 늘어나면 Raw 코드는 흩어집니다

읽는 방법뿐 아니라 결과 구조도 매번 달라집니다.

원본	Raw Python에서 필요한 것	결과 구조 문제
TXT	open(), read()	source를 직접 붙여야 함
CSV	csv / pandas	행 단위를 직접 Document화
PDF	pypdf / pdf parser	page 정보 직접 관리
Web	requests + BeautifulSoup	본문·메뉴·광고 분리 필요

Raw 방식의 진짜 불편함 5가지

코드가 짧으냐가 아니라 RAG 파이프라인에서 계속 유지 가능한지가 핵심입니다.

1. 출처 추적

답변 근거를 source/page로 보여주기 어렵습니다.

2. 형식별 코드

TXT, CSV, PDF, Web을 각각 따로 처리해야 합니다.

3. metadata 규칙

어떤 키를 넣을지 팀마다 코드마다 달라질 수 있습니다.

4. 후속 처리 연결

Splitter, Embedding, VectorStore로 넘길 구조를 직접 맞춰야 합니다.

5. 디버깅

검색 결과가 이상할 때 원본 위치를 추적하기 어렵습니다.

DocumentLoader와 Document 객체

- Loader는 단순 읽기 도구가 아니라 표준화 도구입니다.
- 모든 문서는 page_content와 metadata를 가집니다.
- Loader 선택은 "검색 단위"를 기준으로 합니다.

DocumentLoader의 역할

다양한 원본을 Document(page_content, metadata) 리스트로 통일합니다.

입력

TXT, CSV, PDF, Web처럼 원본 형식은 다양합니다.

Loader

각 형식에 맞게 읽고 metadata를 함께 구성합니다.

출력

Document 객체 리스트로 통일됩니다.

핵심

읽기보다 중요한 것은 RAG에서 재사용 가능한 구조로 표준화하는 것입니다.

Document 객체: page_content와 metadata

LangChain에서 문서는 텍스트와 출처 정보를 함께 가진 객체입니다.

page_content

LLM이 읽을 실제 텍스트입니다. TextSplitter는 이 값을 기준으로 문서를 나눕니다.

metadata

source, page, row, title 같은 부가 정보입니다. 출처 표시와 디버깅에 중요합니다.

왜 중요한가?

RAG에서 정답처럼 보이는 답변보다 중요한 것은 근거를 추적할 수 있는 구조입니다.

TextLoader 실습 파일 읽기

Raw Python의 open()과 목적은 비슷하지만 결과가 Document 객체로 나옵니다.

실습 파일: 02_DocumentLoader_TextLoader.py

```
from langchain_community.document_loaders import TextLoader

loader = TextLoader("./data/Sample_Text.txt", encoding="utf-8")
docs = loader.load()

print("docs type:", type(docs))
print("doc type:", type(docs[0]))
print("metadata:", docs[0].metadata)
print("page_content:", docs[0].page_content[:300])
```

코드 설명

TextLoader는 txt/md 같은 단순 텍스트에 적합합니다.

loader는 읽을 준비를 하는 객체입니다.

load()가 실제 로드입니다.

결과는 Document 객체가 들어 있는 리스트입니다.

CSVLoader 실습 파일 읽기

CSV는 보통 행(row) 하나가 하나의 Document가 됩니다.

실습 파일: 03_DocumentLoader_CSVLoader.py

```
from langchain_community.document_loaders import CSVLoader

loader = CSVLoader(file_path="./data/Sample_CSV.csv", encoding="utf-8")
rows = loader.load()

print(f"총 {len(rows)}개 Document")
print(rows[0].metadata)
print(rows[0].page_content)
```

언제 쓰나?

FAQ 목록

상품 목록

고객 문의 데이터

행 하나가 검색 단위가 될 때

DocumentLoader 선택 기준

파일 확장자보다 “검색 단위와 metadata”를 기준으로 고릅니다.

상황	추천 Loader	선택 이유
TXT / MD	TextLoader	파일 전체를 하나의 문서로 읽기 좋음
FAQ / 표 데이터	CSVLoader	행 하나가 검색 단위일 때 적합
PDF 문서	PyPDFLoader	페이지별 Document와 page metadata 필요
웹페이지	WebBaseLoader	URL에서 본문을 가져오되 품질 확인 필요
다양한 형식	Unstructured 계열	문서 종류가 섞여 있을 때 검토

PDF Loader와 출처 추적

- PDF는 RAG에서 가장 많이 쓰이지만 추출 품질 문제가 많습니다.
- page_content 품질과 metadata를 반드시 확인합니다.
- WebBaseLoader는 보충 실습으로 다룹니다.

PyPDFLoader 실습 파일 읽기

PDF는 보통 페이지 단위 Document 리스트로 로드됩니다.

실습 파일: 04_DocumentLoader_PyPDFLoader.py

```
from langchain_community.document_loaders import PyPDFLoader

loader = PyPDFLoader("./data/SPRi AI Brief_10월호_산업동향_1002_F.pdf")
pages = loader.load()

print(len(pages))
print(pages[0].metadata)
print(pages[0].page_content[:500])
```

확인할 것

PDF가 몇 개의 Document로 로드되었는가?

metadata에 source/page가 있는가?

page_content가 검색에 쓸 만한가?

PDF Loader에서 꼭 확인할 것

읽혔는지가 아니라 RAG에 쓸 만하게 추출됐는지가 중요합니다.

실습 파일: 04_DocumentLoader_PyPDFLoader.py

```
for i, page in enumerate(pages[:3]):  
    print("=" * 50)  
    print(f"페이지 인덱스: {i}")  
    print(page.metadata)  
    print(page.page_content[:300])
```

체크리스트

페이지 수가 예상과 맞는가?

source/page metadata가 있는가?

텍스트 줄바꿈이 지나치게 깨지지 않았는가?

스캔 PDF라면 OCR이 필요할 수 있습니다.

표 중심 문서는 별도 파서를 고려합니다.

PyPDFLoader 특징

PDF는 보통 페이지 단위 Document 리스트로 로드됩니다.

실습 파일: 04_DocumentLoader_PyPDFLoader.py

항목	설명
내부 라이브러리	pypdf
장점	설치가 단순하고, 기본 PDF 텍스트 추출에 적합
용도	논문, 보고서, 교재처럼 텍스트가 비교적 깔끔한 PDF
한계	복잡한 표, 2단 레이아웃, 스캔 PDF에서는 결과가 깨질 수 있음

PyMUPDF 실습 파일 읽기

PDF는 보통 페이지 단위 Document 리스트로 로드됩니다.

실습 파일: 04-02_DocumentLoader_PyMuPDF.py

```
from langchain_community.document_loaders import PyMuPDFLoader

loader = PyMuPDFLoader(pdf_filepath)
pages = loader.load()
```

PyMuPDFLoader 특징

PDF는 보통 페이지 단위 Document 리스트로 로드됩니다.

실습 파일: 04-02_DocumentLoader_PyMuPDF.py

항목	설명
내부 라이브러리	PyMuPDF
장점	빠르고, PDF 구조/위치/이미지/표 처리 쪽 기능이 더 강함
용도	표, 이미지, 복잡한 레이아웃이 있는 PDF
RAG 관점	PyMuPDF를 쓰면 Markdown 형태 추출도 가능해서 RAG 전처리에 유리할 수 있음

WebBaseLoader 실습 파일 읽기

웹페이지도 Document로 가져올 수 있지만 본문 추출 품질을 확인해야 합니다.

실습 파일: 05_DocumentLoader_WebBaseLoader.py

```
from langchain_community.document_loaders import WebBaseLoader

loader = WebBaseLoader(
    "https://python.langchain.com/docs/introduction/",
    bs_get_text_kwargs={"separator": " ", "strip": True},
)
docs = loader.load()

print(docs[0].metadata)
print(docs[0].page_content[:1000])
```

운영 포인트

웹은 네트워크 환경에 따라 실패할 수 있습니다.

메뉴, 광고, 버튼 텍스트가 섞일 수 있습니다.

본문 추출 품질을 반드시 확인해야 합니다.

본편 필수보다 보충 실습으로 운영합니다.

TextSplitter란?

로드된 문서를 LLM이 처리하기 좋은 작은 조각(chunk)으로 나누는 과정.

크고 복잡한 문서를 LLM이 받아들일 수 있는 효율적인 작은 규모의 조각으로 나누는 작업

시스템이 정보를 보다 잘 활용할 수 있도록 나누는 작업.

목적 : 나중에 사용자가 입력한 질문에 대하여 보다 효율적인 정보만 압축/선별하여 가져오기 위함

왜 문서를 나눠야 할까?

TextSplitter는 검색 품질과 답변 품질을 함께 좌우합니다.

토큰 제한

LLM은 한 번에 입력 가능한 길이에 한계가 있습니다.

문맥 유지

너무 작으면 문맥이 부족하고, 너무 크면 노이즈가 많습니다.

검색 단위

질문과 관련 있는 부분만 찾으려면 적절한 단위가 필요합니다.

리소스 최적화(효율성)

답변에 필요한 정보만 발췌.

좋은 chunk와 나쁜 chunk

chunk는 RAG에서 실제로 검색되는 최소 단위입니다.

너무 작은 chunk

검색 단위는 세밀하지만 문맥이 부족합니다. 정책명과 시행일이 서로 다른 chunk로 갈라질 수 있습니다.

적절한 chunk

질문에 답하는 데 필요한 제목, 설명, 조건이 함께 들어 있습니다.

너무 큰 chunk

문맥은 많지만 관련 없는 내용도 함께 들어와 검색 결과가 노이즈를 포함할 수 있습니다.

Raw Python: 문자열 직접 자르기

단순 slicing은 가능하지만 문장과 문단 경계를 고려하지 못합니다.

실습 파일: 06_raw_textSplitter.py

```
chunk_size = 50
chunks = []

for i in range(0, len(text), chunk_size):
    chunk = text[i:i + chunk_size]
    chunks.append(chunk)

for i, chunk in enumerate(chunks):
    print(f"[{i+1}] {chunk}")
```

불편한 점

문장 중간에서 잘릴 수 있습니다.
단어가 끊길 수 있습니다.
metadata가 없습니다.
문맥 경계를 고려하지 않습니다.

CharacterTextSplitter란?

텍스트를 일정한 크기로 분할할 수 있는 간단한 도구.

정의된 구분자를 사용하여 텍스트를 나눈다

주로 특정 문자를 기준으로 분할하기 때문에, 문장이나 문단 단위로 텍스트를 나누는데 효과적

CharacterTextSplitter 특징

두 설정은 RAG 품질을 좌우하는 핵심 하이퍼파라미터입니다.

기본 구분자

기본은 `wnwn`

단순하고 직관적

사용자가 설정한 구분자에 따라 텍스트를 분리

길이 제한 가능

사용자가 원하는 길이 제한을 설정

간단하고 빠르고 직관적인 분할이 필요할 때 유용.

chunk_size와 chunk_overlap

두 설정은 RAG 품질을 좌우하는 핵심 하이퍼파라미터입니다.

chunk_size

한 chunk의 최대 크기입니다. 작으면 세밀하지만 문맥이 부족하고, 크면 문맥은 많지만 노이즈가 늘 수 있습니다.

chunk_overlap

앞뒤 chunk가 겹치는 길이입니다. 경계에서 문맥이 끊기는 문제를 줄여줍니다.

운영 감각

정답 숫자는 없습니다. 문서 성격, 질문 유형, 모델 context 길이에 맞춰 실험합니다.

CharacterTextSplitter 실습

지정한 separator를 기준으로 텍스트를 나눕니다.

실습 파일: 07_Text Splitter_CharacterTextSplitter.py

```
from langchain_text_splitters import CharacterTextSplitter

text_splitter = CharacterTextSplitter(
    separator=" \n\n",
    chunk_size=50,
    chunk_overlap=10,
)

chunks = text_splitter.split_text(sample_text)
```

언제 좋나?

분할 기준이 명확할 때
빈 줄, 구분자, 특정 기호 중심 문서
TextSplitter 개념 이해용

RecursiveCharacterTextSplitter란?

CharacterTextSplitter보다 한 단계 더 발전된 방식

Text를 보다 유연하게 분할

텍스트가 설정한 최대 길이를 초과할 경우 더 작은 단위로 나눌 수 있음.

RecursiveCharacterTextSplitter 특징

두 설정은 RAG 품질을 좌우하는 핵심 하이퍼파라미터입니다.

다중 구분자 사용

여러 구분자를 사용하여 텍스트를 나눔
문단 -> 줄바꿈 -> 공백 -> 글자

너무 긴 조각만 다시 나눔

Chunk_size를 넘으면, 그 문단만 줄바꿈이나 공백 같은 더 작은 기준으로 다시 나눔.

문맥을 최대한 유지

다양한 구분자를 사용해 문맥을 더 잘 유지

긴 텍스트를 다루며 문맥을 유지하고 복잡한 텍스트 구조를 다룰 때.

RecursiveCharacterTextSplitter 실습

문단, 문장, 단어 순서로 최대한 자연스럽게 나누려고 시도합니다.

실습 파일: 08_Text Splitter_RecursiveCharacterTextSplitter.py

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=100,
    chunk_overlap=20,
)

chunks = text_splitter.split_text(sample_text)
```

왜 기본으로 좋나?

일반 PDF/TXT RAG의 좋은 출발점입니다.
문맥 경계를 최대한 유지하려고 합니다.
chunk_size를 바꿔가며 실험하기 좋습니다.

PDF 문서를 split_documents로 나누기

Document 리스트를 나누면 metadata가 유지됩니다.

실습 파일: 09_Text Splitter_RecursiveCharacterTextSplitter_pdf.py

```
loader = PyPDFLoader("./data/SPRi AI Brief_10월호_산업동향_1002_F.pdf")
pages = loader.load()

splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=50,
)
chunks = splitter.split_documents(pages)

print(len(chunks))
print(chunks[0].metadata)
print(chunks[0].page_content[:500])
```

중요한 이유

split_text는 문자열만 나눕니다.
split_documents는 Document를 나눕니다.
metadata가 유지됩니다.
Day 3 출처 표시와 연결됩니다.

split_text vs split_documents

문자열만 나눌지 metadata를 가진 Document를 나눌지 구분해야 합니다.

구분	입력	출력	사용 상황
split_text	문자열	문자열 리스트	간단한 샘플 실습
split_documents	Document 리스트	Document 리스트	실제 RAG 문서 처리
핵심 차이	metadata 없음/있음	source/page 유지 여부	출처 표시와 디버깅

TokenTextSplitter란?

글자 수 기준이 아니라 token 기준으로 텍스트를 자르는 splitter

TokenTextSplitter특징

글자 수가 아니라 token 수 기준으로 chunk를 만든다

"LangChain은 좋은 도구입니다."
↓
[토큰ID, 토큰ID, 토큰ID, 토큰ID, ...]

Text를 tokenizer로 encode 뒤 다시 decode

```
TokenTextSplitter(  
    chunk_size=100,  
    chunk_overlap=10  
)
```

자른 token ID를 다시 문자열로 되돌린다

원본 텍스트
↓
tokenizer로 token ID 변환
↓
token ID를 chunk_size 단위로 자름
↓
자른 token ID를 다시 문자열로 decode
↓
chunk 리스트 반환

문서 구조보다 token 제한이 더 중요한 경우에 사용

TokenTextSplitter특징

주의

- TokenTextSplitter의 차이는 이 값들이 token 기준으로 적용된다는 점이다.
- 한국어/중국어/일본어처럼 토큰화가 복잡한 언어에서는 결과를 확인해야 한다.

⇒ RecursiveCharacterTextSplitter.from_tiktoken_encoder()

⇒ CharacterTextSplitter.from_tiktoken_encoder()

문서 종류에 따른 Splitter 선택 기준

문서 유형	추천 Splitter	선택 이유
일반 PDF / 텍스트	RecursiveCharacterTextSplitter	문단, 줄바꿈, 공백 기준으로 문맥 보존
토큰 제한이 중요한 문서	Token 기준 Splitter	LLM은 글자 수가 아니라 token 수 기준
Markdown	MarkdownHeaderTextSplitter	제목 구조와 섹션 정보 보존
Code	코드용 Splitter / from_language()	함수, 클래스 구조 보존
의미 단위 분할	SemanticChunker	의미 흐름 기준 분할, 비용·속도 부담 있음

TokenTextSplitter: 토큰화 결과 직접 보기

같은 문장도 tokenizer와 모델 계열에 따라 token 수가 달라질 수 있습니다.

실습 파일: 10_token_example.py

```
import tiktoken

text = "나는 LangChain을 좋아합니다."

encoding_cl100k = tiktoken.get_encoding("cl100k_base")
encoding_o200k = tiktoken.get_encoding("o200k_base")

print(len(encoding_cl100k.encode(text)))
print(len(encoding_o200k.encode(text)))
```

간단 설명

cl100k_base: GPT-3.5/일부 GPT-4 계열에서 많이 쓰인 토큰화 규칙
o200k_base: GPT-4o 계열 이후에서 쓰이는 더 큰 vocabulary 기반 규칙
중요한 것은 token 번호가 아니라 token 수 감각입니다.

TokenTextSplitter 실습

LLM 입력 길이와 비용을 token 기준으로 관리할 때 사용합니다.

실습 파일: 11_Text Splitter_TokenTextSplitter.py

```
from langchain_text_splitters import TokenTextSplitter

text_splitter = TokenTextSplitter(
    chunk_size=15,
    chunk_overlap=5,
    encoding_name="o200k_base",
)

chunks = text_splitter.split_text(sample_text)
```

언제 쓰나?

토큰 제한을 엄격히 맞춰야 할 때
긴 문서 요약
비용 계산이 중요한 작업
모델 context window에 맞춰 chunk를 관리할 때

영어와 한글 토큰화 확인하기

겉보기 글자 수와 token 수는 다를 수 있습니다.

실습 파일: 12_Text Splitter_TokenTextSplitter_korean_english.py

```
encoding = tiktoken.get_encoding("o200k_base")

english_text = "I love LangChain"
korean_text = "나는 랭체인을 좋아합니다"

english_tokens = encoding.encode(english_text)
korean_tokens = encoding.encode(korean_text)

print(len(english_tokens))
print(len(korean_tokens))
```

확인할 것

글자 수와 token 수는 다릅니다.
한글, 영어, 숫자, 기호에 따라 token 수가 달라집니다.
token id 자체보다 token 수를 확인하는 습관이 중요합니다.

MarkdownHeaderTextSplitter

제목 구조가 중요한 문서는 header 기준으로 나누는 것이 자연스럽습니다.

실습 파일: 13_Text Splitter_MarkdownHeaderTextSplitter.py

```
from langchain_text_splitters import MarkdownHeaderTextSplitter

headers_to_split_on = [
    ("#", "Header 1"),
    ("##", "Header 2"),
    ("###", "Header 3"),
]

splitter = MarkdownHeaderTextSplitter(headers_to_split_on)
chunks = splitter.split_text(markdown_text)
```

언제 쓰나?

README.md
API 문서
위키 문서
제목과 소제목이 의미 단위일 때
섹션 제목을 metadata처럼 보존하고 싶을 때

CodeSplitter

코드 문서는 함수와 클래스 구조가 끊기지 않도록 나누는 것이 중요합니다.

실습 파일: 14_Text Splitter_CodeSplitter.py

```
from langchain_text_splitters import RecursiveCharacterTextSplitter, Language

splitter = RecursiveCharacterTextSplitter.from_language(
    language=Language.PYTHON,
    chunk_size=300,
    chunk_overlap=50,
)
chunks = splitter.split_text(python_code)
```

언제 쓰나?

소스코드 RAG
GitHub 저장소 분석
함수/클래스 단위 검색
레거시 코드 설명 챗봇

SemanticChunker는 언제 쓸까?

의미 변화 지점을 기준으로 나누는 심화 splitter입니다.

실습 파일: 15_Text Splitter_SemanticChunker.py

```
from langchain_experimental.text_splitter import SemanticChunker
from langchain_openai import OpenAIEmbeddings

splitter = SemanticChunker(
    OpenAIEmbeddings(model="text-embedding-3-small")
)
chunks = splitter.split_text(long_text)
```

주의할 점

langchain-experimental 설치가 필요합니다.
chunking 과정에서 embedding 모델을 사용합니다.
API 비용과 속도 이슈가 있습니다.
본편 필수보다 심화/선택 실습으로 운영합니다.

Embedding으로 의미 검색 준비하기

- 문자열 비교만으로는 의미 검색이 어렵습니다.
- 문서와 질문을 같은 벡터 공간으로 바꿉니다.
- 간단한 유사도 검색을 직접 맞춥니다.

Embedding을 쉽게 이해하기

의미를 숫자 좌표로 바꿔 가까운 문장을 찾는 방식입니다.

문장

"저는 강아지를 좋아합니다."
"제가 좋아하는 동물은 무엇인가요?"

Embedding

문장을 1536차원 같은 숫자 배열로 변환합니다.
[0.012, -0.084, ...]

유사도

질문 벡터와 문서 벡터가 가까우면 관련 문서로 판단합니다.

비유

문장의 의미를 지도 위 좌표로 바꾸는 것과 비슷합니다.

OpenAIEmbeddings 실습

문서 여러 개는 embed_documents, 질문 하나는 embed_query를 사용합니다.

실습 파일: 16_Embedding_OpenAIEmbeddings.py

```
from langchain_openai import OpenAIEmbeddings

embeddings_model = OpenAIEmbeddings(
    model="text-embedding-3-small"
)

document_embeddings = embeddings_model.embed_documents(documents)
query_embedding = embeddings_model.embed_query(query)
```

코드 설명

embed_documents: 여러 문서/chunk를 벡터로 변환
embed_query: 사용자 질문 하나를 벡터로 변환
Day 3에서는 문서 벡터를 PGVector에 저장합니다.

embed_documents vs embed_query

문서 저장용과 질문 검색용의 역할을 구분합니다.

구분	사용 시점	입력	결과
embed_documents	문서 저장 전	문서/chunk 리스트	벡터 리스트
embed_query	질문이 들어올 때	질문 문자열 하나	질문 벡터 하나
RAG 흐름	문서는 미리 저장, 질문은 요청마다 변환	문서 vs 질문	가까운 chunk 검색

PDF 페이지 임베딩 실습

PDF를 로드한 뒤 page_content 일부만 embedding합니다.

실습 파일: 17_Embedding_PDF_Pages.py

```
sample_docs = documents[:3]
document_texts = [doc.page_content for doc in sample_docs]

document_embeddings = embeddings_model.embed_documents(document_texts)

print(len(document_embeddings))
print(len(document_embeddings[0]))
print(document_embeddings[0][:5])
```

왜 3개만?

Embedding도 API 호출입니다.
전체 PDF를 바로 임베딩하면 비용과 시간이 늘어납니다.
처음에는 일부만 테스트하는 습관이 좋습니다.
벡터 전체가 아니라 길이와 앞 5개 값만 확인합니다.

Embedding 모델 선택 기준

모델은 차원, 비용, 속도, 언어 품질, 운영 환경을 함께 보고 고릅니다.

OpenAIEmbeddings

빠른 실습과 API 기반 서비스에 적합합니다. 설치가 단순하지만 비용이 발생합니다.

HuggingFaceEmbeddings

폐쇄망, 온프레미스, 비용 민감 환경에서 고려합니다. 모델 다운로드와 성능 확인이 필요합니다.

운영 서비스

차원이 바뀌면 기존 벡터를 재색인해야 할 수 있습니다. 캐싱 전략이 중요합니다.

한국어 품질

다국어/한국어 임베딩 모델을 비교하고 검색 결과로 판단해야 합니다.

간단한 유사도 검색 맛보기

Day 3 PGVector가 할 일을 작은 예제로 직접 확인합니다.

실습 파일: 18_Embedding_similarity_demo.py

```
documents = [
    "AI 반도체 시장이 빠르게 성장하고 있습니다.",
    "생성형 AI 서비스가 기업 업무에 도입되고 있습니다.",
    "오늘 점심 메뉴는 김치찌개입니다.",
]
query = "인공지능 산업 동향은 어떤가요?"

# 1. 문서 embedding
# 2. 질문 embedding
# 3. cosine similarity 계산
# 4. 가장 가까운 문장 확인
```

실습 목표

Embedding이 단순 숫자 배열이 아니라 검색을 위한 준비임을 확인합니다.
질문과 가까운 문서가 무엇인지 직접 예상해봅니다.
Day 3의 VectorStore 검색으로 자연스럽게 연결합니다.

Day 2 전체 파이프라인 연결

- DocumentLoader가 문서를 읽습니다.
- TextSplitter가 검색 가능한 chunk로 나눕니다.
- Embedding이 chunk를 벡터로 바꿉니다.
- Day 3에서는 이 벡터를 PGVector에 저장합니다.

Day 2 파이프라인 지도

PDF 한 개가 RAG 챗봇 지식 베이스가 되기까지의 흐름입니다.

1. Load

```
PyPDFLoader  
pages = loader.load()
```

2. Split

```
RecursiveCharacterTextSplitter  
chunks = split_documents(pages)
```

3. Embed

```
OpenAIEmbeddings  
embed_documents(texts)
```

4. Store

```
PGVector  
Day 3에서 저장하고 검색
```

Day 2 핵심 정리

오늘 배운 세 단계가 Day 3 RAG 챗봇의 입력이 됩니다.

DocumentLoader

원본 파일을 Document 객체로 변환합니다.

TextSplitter

Document를 검색 가능한 chunk로 나눕니다.

Embedding

chunk와 질문을 의미 벡터로 변환합니다.

오늘의 문장

RAG는 모델만의 문제가 아닙니다. 문서를 어떻게 읽고, 나누고, 벡터화하느냐가 답변 품질을 결정합니다.

Day 3 예고

내일은 오늘 만든 지식 베이스를 PGVector에 저장하고 RAG 챗봇 v1을 만듭니다.

VectorStore

Embedding vector와 원문 Document를 함께 저장합니다.

Retriever

질문을 받아 관련 chunk를 찾아주는 검색기입니다.

RAG Chain

검색된 context를 Prompt에 넣고 모델이 답변합니다.

연결 문장

오늘 만든 pages, chunks, embeddings가 내일 PGVector와 Retriever로 연결됩니다.

16:00~18:00 오늘의 과제

- 과제명: RAG 챗봇용 지식 베이스 준비하기
- PDF를 로드하고 chunking하고 embedding까지 확인합니다.
- 길고 어려운 리포트가 아니라 실행과 관찰이 목표입니다.

Day 2 과제 요구사항

기존 리포트 과제를 줄이고, chunk_size 변화와 embedding 확인에 집중합니다.

학생이 하는 일

assignments/day2_build_knowledge_base_starter.py를 실행합니다.
PDF가 몇 개의 Document로 로드되는지 확인합니다.
CHUNK_SIZE를 1000에서 500으로 바꿔봅니다.

확인할 것

chunk 개수가 어떻게 달라졌는지 확인합니다.
첫 번째 chunk 1개만 embedding합니다.
vector 길이를 확인합니다.

과제에서 학생이 수정하는 부분

처음에는 1000으로 실행하고, 그다음 500으로 바꿔 chunk 개수만 비교합니다.

실습 파일: `assignments/day2_build_knowledge_base_starter.py`

```
# 학생 수정 영역
PDF_PATH = Path("./data/SPRI AI Brief_10월호_산업동향_1002_F.pdf")

CHUNK_SIZE = 1000
CHUNK_OVERLAP = 100

# 1차 실행: CHUNK_SIZE = 1000
# 2차 실행: CHUNK_SIZE = 500
# chunk 개수 변화를 확인합니다.
```

평가 기준

PDF가 로드되었는가?
metadata를 확인했는가?
CHUNK_SIZE를 한 번 바꿔봤는가?
chunk 개수 변화를 확인했는가?
embedding vector 길이를 확인했는가?